NetML Project: Malware Detection from Raw PCAPs via Flow Statistics

Jeffrey Liu

December 12,2025

1 Introduction

Practical malware detection from network traces is valuable for security operations centers (SOCs), especially when payloads are encrypted or unavailable. This project evaluates classical ML baselines on flow-level features computed from raw PCAPs, mirroring the netML/pcapML approach: packets are aggregated per flow to fixed-length vectors (e.g., sizes, inter-arrival times, flags, ports, and burstiness summaries).

We focus on three questions:

(i) How do simple tabular models perform on binary and multiclass family classification?

(ii) Do models generalize beyond family-specific signature cues (family-held-out evaluation)?

(iii) What is the accuracy–cost trade-off (time/memory/model size) across configurations?

2 Data and Features

Dataset. netML Malware Detection (Stratosphere IPS), consisting of benign and malware traffic traces in PCAP format.

Feature pipeline. We use flow/time-window statistics ("flowstats") aggregated into a fixed-length feature vector per PCAP/flow. In our current configuration, we use:

- $N = 10$ packet/flow aggregation setting ("flowstats_N10")
- **39** numeric features per example
- One example row per input file/flow (see sanity checks below)

Sanity checks for leakage / duplication. We verified:

- `unique_files` equals number of rows (one row per file)
- Exact-duplicate feature rows exist but are limited (about $\approx 2.25\%$ of rows), so we avoid attributing strong performance solely to duplicated entries split across folds.
- A shuffled-label baseline on the binary task yields balanced accuracy and ROC-AUC near 0.5, consistent with no trivial leakage.

3 Models

We evaluate the following models:

- Logistic Regression (LogReg), class-weight balanced
- Linear SVM (LinearSVM)
- RFF Kernel SVM approximation (Kernel-SVM (RFF))
- Random Forest (RandomForest)

4 Experimental Setup

4.1 Tasks

- **Easy (binary):** $y_{\text{easy}} \in \{0, 1\}$ for benign vs. malware.
- **Hard (19-class):** $y_{\text{hard}} =$ `family` (18 malware families + benign).

4.2 Metrics

Primary metric: **Balanced Accuracy (BA)**. Secondary metrics:

- Binary: F1, ROC-AUC
- Multiclass: Macro-F1; confusion matrices

4.3 Cross-validation

We report 5-fold stratified CV means and standard deviations where applicable.

4.4 Robustness: Family-held-out evaluation (binary)

For each malware family F:

- Train on: all benign + all malware families *except* F
- Test on: all benign + F

We then convert predictions/labels to binary by mapping `benign` $\rightarrow 0$ and all malware families $\rightarrow 1$, and report mean BA across held-out families.

4.5 Systems-cost instrumentation

For each configuration we measure:

- feature file size on disk (MB)
- load time, fit time, predict time (seconds), plus total
- CPU utilization estimate (process CPU percent over interval)
- peak RSS delta (MB) and end RSS (MB)
- serialized model artifact size (MB)

We then plot accuracy vs total wall-clock time for discrete sweeps and mark Pareto-efficient points.

5 Results

5.1 Binary task (5-fold CV)

Table 1 summarizes 5-fold CV on the binary task.

Table 1: Easy (binary) 5-fold CV results.

Model	Balanced Acc (mean ± std)	F1 (mean ± std)
LogReg	0.9379 ± 0.0010	0.9511 ± 0.0007
LinearSVM	0.9360 ± 0.0007	0.9478 ± 0.0005
Kernel-SVM (RFF)	0.9682 ± 0.0002	0.9744 ± 0.0001
RandomForest	0.9989 ± 0.0001	0.9994 ± 0.0001

5.2 Hard 19-class task (5-fold CV)

Table 2 summarizes 5-fold CV on the 19-class task (macro-F1 and BA).

Table 2: Hard (19-class) 5-fold CV results.

Model	Balanced Acc (mean ± std)	Macro-F1 (mean ± std)
LogReg	0.5140 ± 0.0014	0.3994 ± 0.0030
LinearSVM	0.4529 ± 0.0050	0.3662 ± 0.0033
Kernel-SVM (RFF)	0.5525 ± 0.0032	0.4654 ± 0.0020
RandomForest	0.6020 ± 0.0025	0.6030 ± 0.0014

5.3 Robustness: Family-held-out (binary)

Table 3 reports mean BA across 18 held-out malware families when testing on $\{\texttt{benign} \cup F\}$ and evaluating after mapping labels to binary.

Table 3: Family-held-out robustness (binary). Mean BA across 18 held-out families.

Model	Mean BA	Min BA	Max BA
RandomForest	0.9874	0.8699	0.9995
LogReg	0.9242	0.7755	0.9800
Kernel-SVM (RFF)	0.9239	0.6113	0.9889
LinearSVM	0.9127	0.5967	0.9802

Interpretation. Family-held-out BA remains high for RandomForest and competitive for LogReg/RFF-SVM, indicating the pipeline is not purely exploiting single-family signature cues; however, the *minimum* BA can drop substantially for certain families (notably for SVM variants), suggesting generalization varies across families.

Model	TN	FP	FN	TP	Balanced Acc.	F1
LogReg	24640	1184	5804	68062	0.938	0.953
LinearSVM	24746	1078	6276	67590	0.937	0.949
Kernel-SVM (RFF)	25316	508	3193	70673	0.968	0.975
RandomForest	25788	36	58	73808	0.999	0.999

Table 4: Binary task confusion-matrix counts and derived metrics on the holdout split. Balanced accuracy is the mean of the benign recall (specificity) and malware recall (sensitivity).

5.4 Confusion matrices

We report confusion matrices for both tasks using the same stratified holdout split used elsewhere in this report. For the binary task, the plotted class labels are `0=benign` and `1=malware`. For the 19-class task, labels correspond to malware families (plus benign variants) as shown on the axes. Because class frequencies are highly non-uniform, raw (count) confusion matrices can be visually dominated by frequent classes; therefore, for the multi-class setting we additionally include *row-normalized* confusion matrices where each row sums to 1, making each diagonal entry directly interpretable as per-class recall.

5.5 Binary task (benign vs. malware)

Figure 1 shows confusion matrices for four models. Across models, false positives (benign predicted as malware) are comparatively rare, while the dominant error mode for the linear models is false negatives (malware predicted as benign). Random Forest achieves near-perfect separation on this split, while the RFF-based kernel SVM reduces both false positives and false negatives relative to the linear baselines.

5.6 Hard task (19-class family classification)

For the 19-class setting, the count-based confusion matrices (Figure 3) show absolute error volume but can understate minority-class failures due to strong class imbalance. To address this, we additionally report row-normalized confusion matrices (Figure 4); these highlight *per-family* performance and make systematic confusions easier to spot.

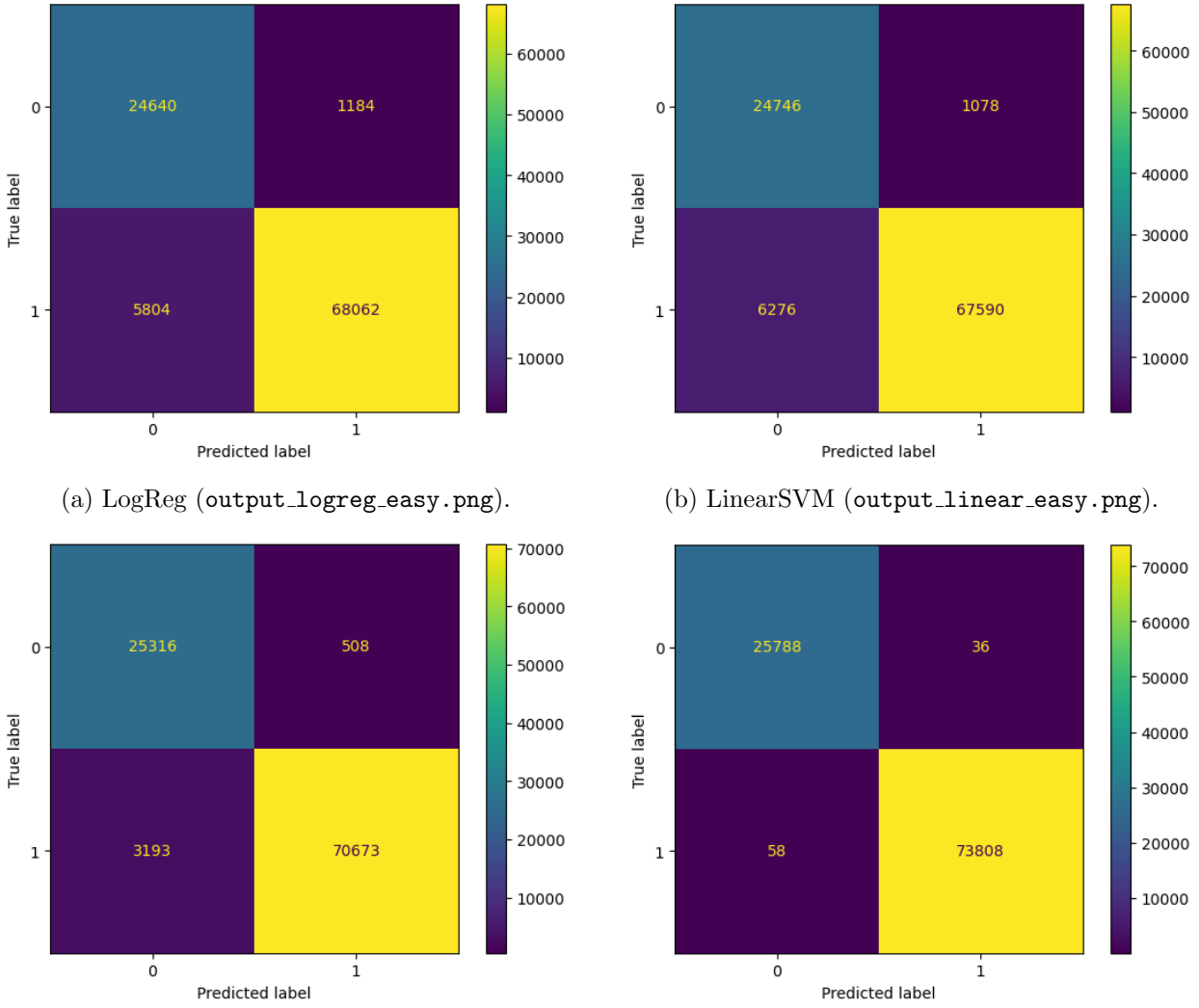
Qualitatively, the Random Forest exhibits a stronger and cleaner diagonal than the linear baseline, indicating higher average per-family recall. The normalized plots also show that remaining errors are concentrated in a small number of family pairs/clusters (e.g., families with similar traffic/flow signatures), rather than being uniformly distributed across all labels. In particular, the `trickbot` vs. `trickster` block exhibits visible cross-confusion, which is consistent with closely related behavioral features at the flow-statistic level. Normalization is therefore essential for interpreting which families are genuinely difficult versus which are simply frequent.

6 Systems Cost Analysis

6.1 Method

We benchmarked end-to-end cost (load + fit + predict) and memory/model sizes for:

- **LogReg sweep:** $C \in \{0.1, 1, 10\}$



(a) LogReg (`output_logreg_easy.png`).

(b) LinearSVM (`output_linear_easy.png`).

(c) Kernel-SVM (RFF) (`output_kernel_easy.png`).

(d) RandomForest (`output_rf_easy.png`).

Figure 1: Binary task confusion matrices (holdout). Axis labels are `0=benign`, `1=malware`.

- **RandomForest sweep:** `n_estimators` $\in \{10, 50, 100\}$

We report discrete points, and interpret the frontier via Pareto efficiency (a point is Pareto if no other point has both lower cost and higher accuracy). Although Kernel-SVM (RFF) improves accuracy over linear baselines on both tasks, it was not practical at our dataset scale: a single end-to-end run (fit + predict on the holdout split) required on the order of hours on our hardware (approximately 3 hours in the longest run). Because our goal includes a cost–accuracy tradeoff study, we treat the SVM results as informative but not deployment-feasible in this setting, and we focus cost curves and recommendations on models/configurations that can be trained and evaluated within reasonable time budgets.

6.2 Key measurements (selected)

Table 5 lists representative results from the sweep (feature file size is constant because only the N10 feature file is available).

Table 5: Selected accuracy–cost points (feature file ≈ 53.75 MB).

Task	Model	Sweep	BA	Fit (s)	Pred (s)	Total (s)	Model (MB)
easy_binary	LogReg	$C = 0.1$	0.8859	35.064	0.012	35.076	0.0009
easy_binary	RandomForest	$n = 10$	0.9991	0.939	0.017	0.956	0.2812
easy_binary	RandomForest	$n = 100$	0.9989	6.649	0.062	6.710	2.8640
hard_19class	LogReg	$C = 10$	0.2480	263.946	0.019	263.966	0.0066
hard_19class	RandomForest	$n = 10$	0.5937	1.895	0.075	1.970	16.0498
hard_19class	RandomForest	$n = 100$	0.5976	10.143	0.376	10.520	160.4889

6.3 Accuracy–cost trade-off and "curve"

Although plots show discrete points (not a smooth line), they still define an empirical accuracy–cost curve over configurations. The Pareto frontier typically includes:

- A **fast, high-accuracy** RandomForest with small/medium number of trees (e.g., $n = 10$)
- A **slightly higher-accuracy but more expensive** RandomForest (e.g., $n = 50$ or $n = 100$)
- LogReg variants are small models but can be much slower on the hard task and achieve lower BA.

Best-value recommendation. Given the measured points, a strong best-value configuration is:

- **Binary:** RandomForest with `n_estimators=10` (near-max BA at very low total time).
- **19-class:** RandomForest with `n_estimators=10` provides most of the achievable BA with far lower time and model size than larger forests.

If model size is a hard constraint, LogReg is smallest on disk, but in our measurements it is substantially slower and less accurate on the 19-class task.

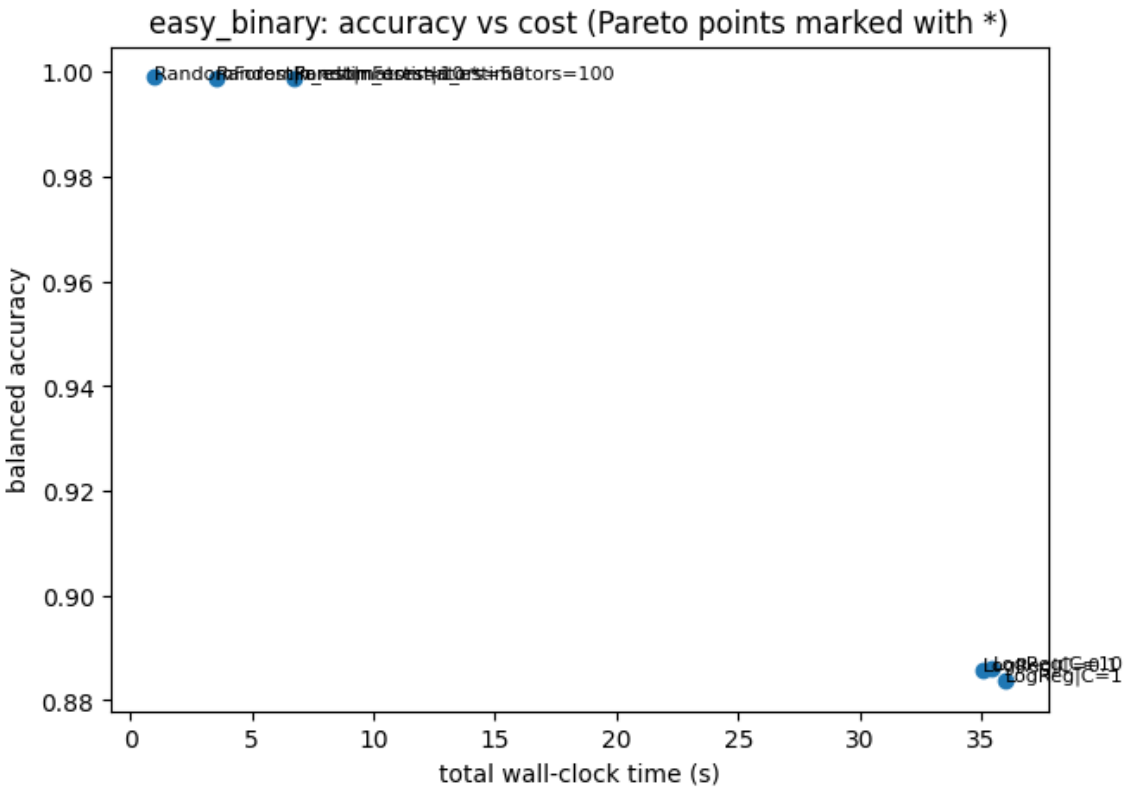


Figure 5: Easy task: balanced accuracy vs. total wall-clock time (discrete sweep points; Pareto marked).

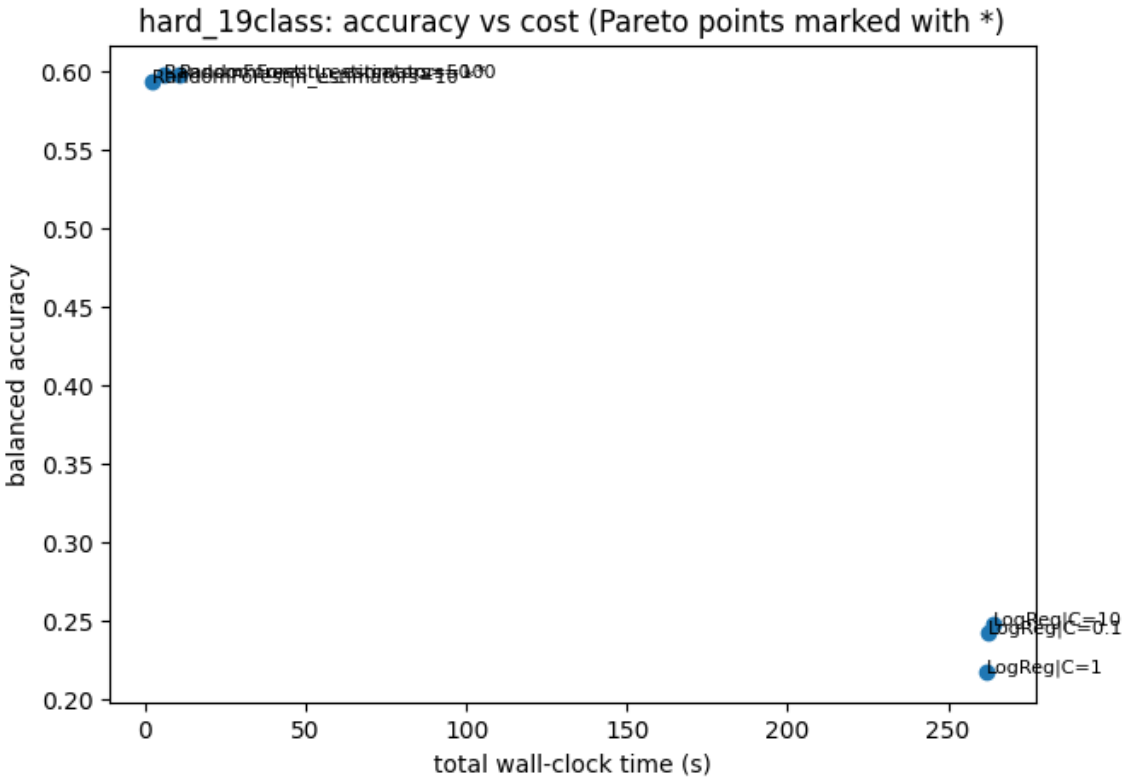


Figure 6: Hard task: balanced accuracy vs. total wall-clock time (discrete sweep points; Pareto marked).

7 Discussion

Binary task. All models achieve strong BA, with RandomForest near-perfect. The shuffled-label baseline (BA/AUC ≈ 0.5) supports that this performance is not due to trivial leakage.

19-class task. Performance drops substantially relative to binary detection, which is expected: multiclass family attribution is harder and class imbalance is more pronounced. RandomForest performs best among tested baselines in both BA and macro-F1.

Robustness. Family-held-out BA remains high for RandomForest, but certain held-out families are more challenging (lower min BA), motivating future work on per-family error analysis and potentially adding domain-robust features.

Systems cost. SVM variants were observed to be computationally expensive in our environment; the cost study and Pareto analysis justify deprioritizing them for "best value" configurations.

8 Limitations and Future Work

- Only one feature configuration (N10) was available in the folder; additional ablations (e.g., N3/N5, reduced headers) would strengthen the cost–accuracy curve across feature pipelines.

- Confusion matrix analysis can be expanded by reporting the top confusions per class and per-family recall/precision.
- More robust evaluation could include temporal splits or host-based splits (if metadata supports it).

9 Conclusion

We reproduced strong binary detection performance and established that multiclass family attribution is substantially harder. Family-held-out evaluation indicates the approach generalizes beyond single-family cues for many families, with RandomForest showing the strongest robustness. A systems-cost study over hyperparameter sweeps provides discrete accuracy–cost trade-offs and supports a best-value recommendation favoring small RandomForests for both tasks.

A Reproducibility Notes

- Random seed: `1337`
- Feature file: `features_flowstats_N10.parquet` (approx. 53.75 MB)
- Export figures from notebook to `figures/` and compile with `pdflatex`.

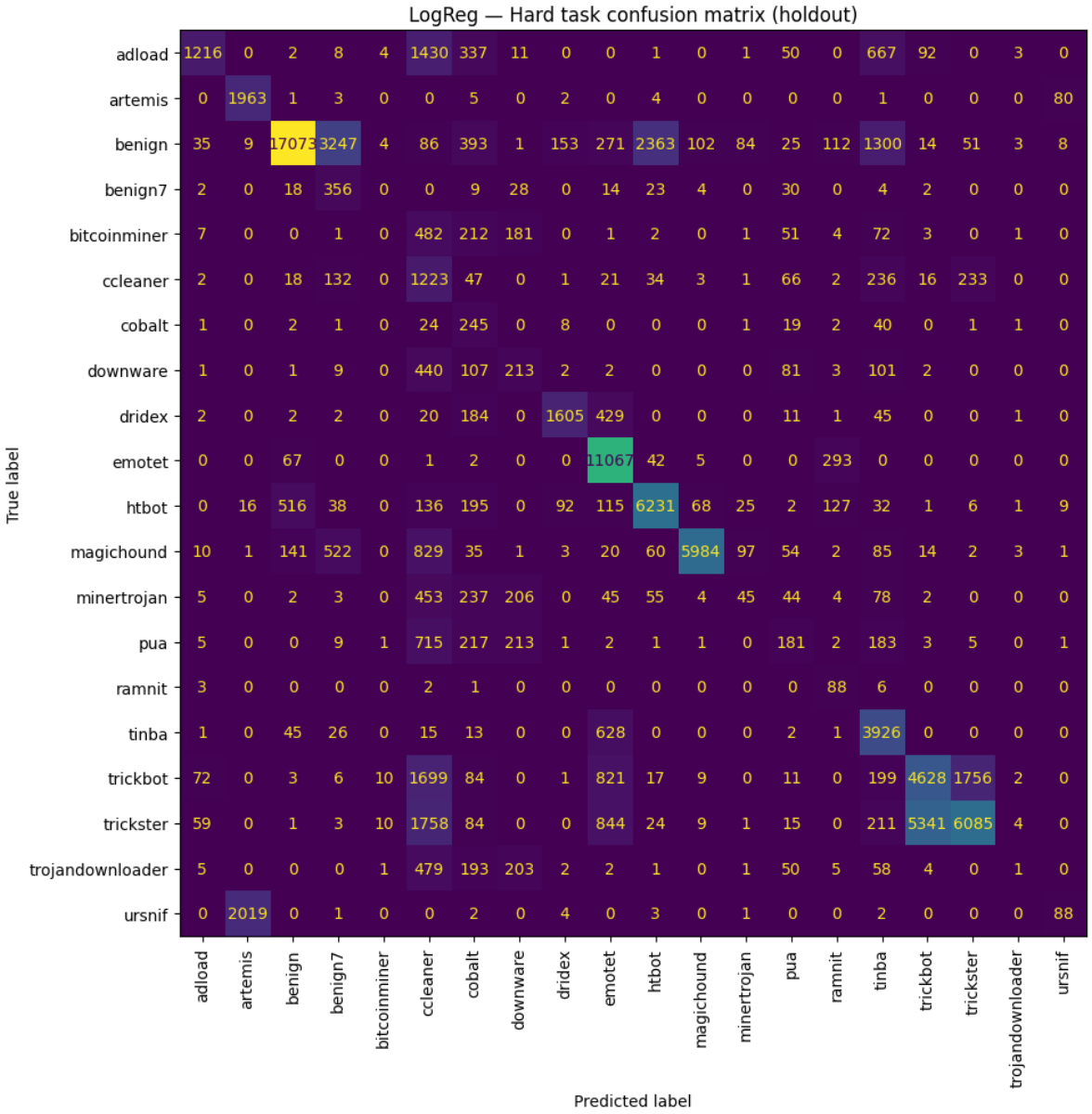
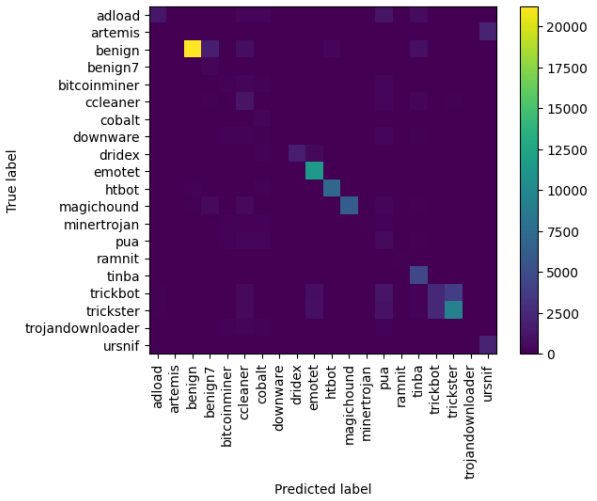
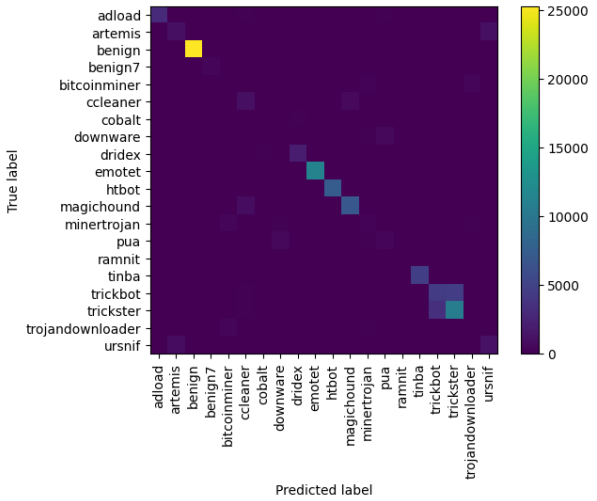


Figure 2: Hard task confusion matrix (counts), Logistic Regression (`output_logreg_hard.png`).

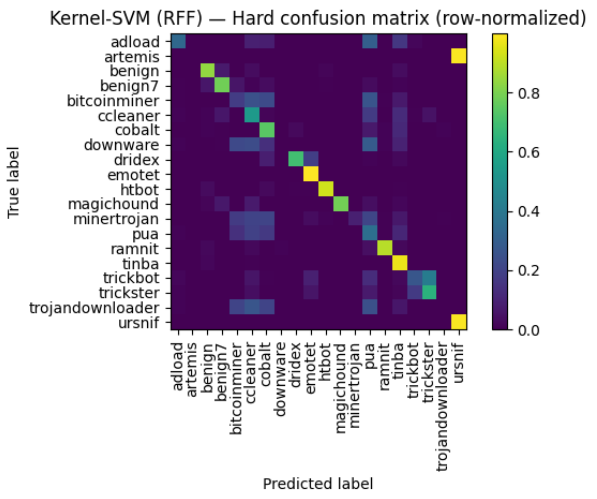


(a) Kernel-SVM (RFF), counts (`output_kernel_hard.png`).

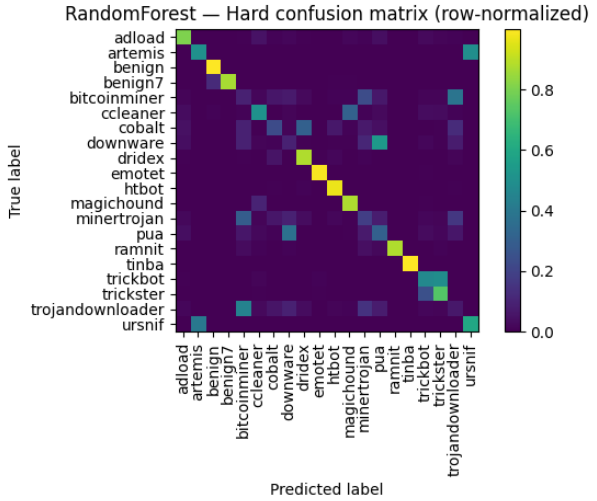


(b) RandomForest, counts (`output_rf_hard.png`).

Figure 3: Hard task confusion matrices (unnormalized counts). See also Figure 4 for row-normalized views.



(a) Kernel-SVM (RFF), row-normalized (`output_kernel_normalized.png`).



(b) RandomForest, row-normalized (`output_rf_normalized.png`).

Figure 4: Hard task row-normalized confusion matrices (each row sums to 1). Diagonal intensity corresponds to per-class recall.